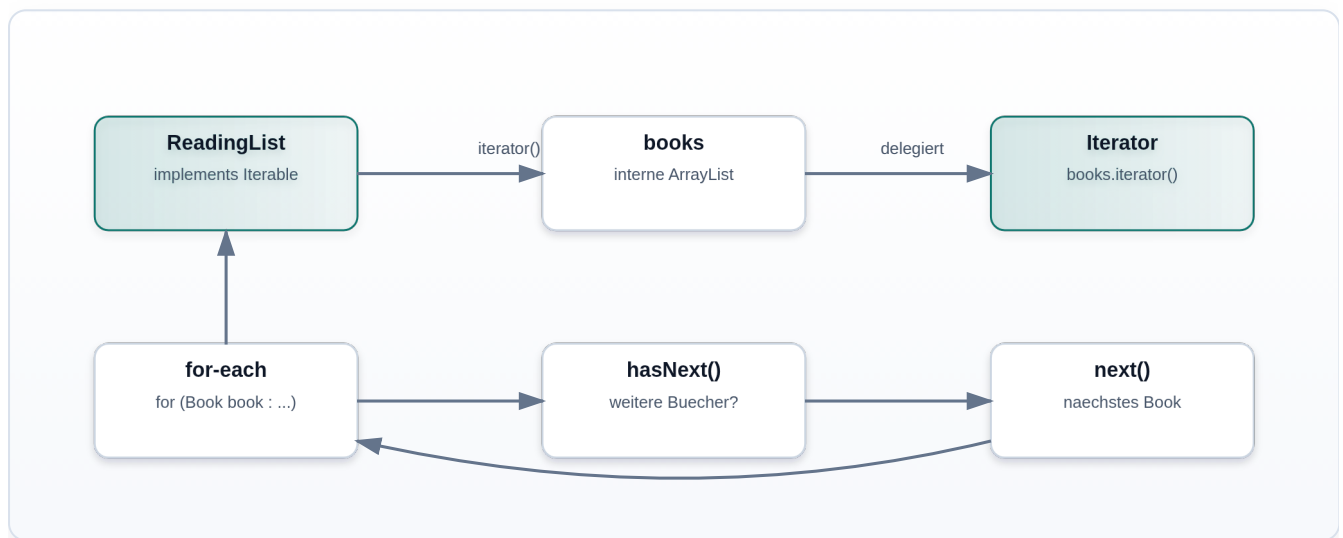


Aufgabe 02: Eine eigene Klasse Iterable machen

Erklärung mit Ablaufdiagramm, zentralem Iterator-Zustand und Bezug zur konkreten Java-Implementierung.

● Professionelles Ablaufdiagramm



■ Grundidee

`ReadingList` ist eine eigene Sammlung von `Book`-Objekten.

Intern speichert die Klasse die Buecher in einer Liste:

```
private final List<Book> books = new ArrayList<>();
```

Von aussen soll `ReadingList` aber wie eine normale Sammlung durchlaufen werden koennen:

```
for (Book book : readingList) {
    System.out.println(book.title());
}
```

Dafuer implementiert `ReadingList` das Interface `Iterable<Book>`.

Was bedeutet Iterable?

Eine Klasse, die `Iterable<Book>` implementiert, verspricht:

Ich kann einen `Iterator<Book>` liefern.

Dieses Versprechen wird durch die Methode `iterator()` eingelöst:

```
@Override
public Iterator<Book> iterator() {
    return books.iterator();
}
```

Damit sagt `ReadingList`:

Wenn jemand durch meine Buecher laufen will, verwende den Iterator meiner internen Liste.

Warum ist die Loesung so kurz?

Die Klasse `ReadingList` muss keinen eigenen Iterator schreiben, weil sie intern bereits eine Java-`List` verwendet.

Eine `List` kann schon einen passenden Iterator liefern:

```
books.iterator()
```

Die Aufgabe besteht deshalb nur darin, diesen vorhandenen Iterator nach aussen weiterzugeben.

Zusammenspiel mit der for-each-Schleife

Eine Enhanced-for-Schleife:

```
for (Book book : readingList) {
    System.out.println(book.title());
}
```

wird vom Compiler sinngemaess wie ein Iterator-Code behandelt:

```
Iterator<Book> iterator = readingList.iterator();
while (iterator.hasNext()) {
    Book book = iterator.next();
    System.out.println(book.title());
}
```

Die `for-each`-Schleife funktioniert also nur, weil `ReadingList` einen Iterator liefern kann.

Warum bleibt die interne Liste gekapselt?

`ReadingList` gibt nicht die ganze interne Liste zurueck. Sie gibt nur einen Iterator zurueck.

Das ist ein wichtiger Unterschied:

- Der Client kann die Buecher durchlaufen.
- Der Client muss nicht wissen, ob intern eine `ArrayList`, `LinkedList` oder eine andere Struktur verwendet wird.
- Die Speicherstruktur kann spaeter geaendert werden, solange `iterator()` weiterhin funktioniert.

Ablauf am Beispiel

`ReadingList.sample()` legt drei Buecher an:

```
Clean Code
Effective Java
Head First Design Patterns
```

Die Demo verwendet:

```
for (Book book : readingList) {
    System.out.println(book.title() + " von " + book.author());
}
```

Java ruft dafuer automatisch `readingList.iterator()` auf.

Der zurueckgegebene Iterator ist der Iterator der internen `books`-Liste. Deshalb werden die Buecher in der Reihenfolge geliefert, in der sie eingefuegt wurden.

Zusammenfassung

Die zentrale Loesung ist:

```
return books.iterator();
```

Damit wird `ReadingList` mit der Enhanced-for-Schleife verwendbar.

Das Beispiel zeigt eine einfache Form des Iterator Patterns: Die Sammlung stellt einen Iterator bereit, und der Client muss die interne Datenstruktur nicht kennen.